VanniKawachh: A Distributed AI Acoustic Intelligence and Autonomous Drone Response Network for Women's Safety

Project Report submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

Mr. Rohan Untawale Mr. Saksham Sabadra
Mr. Rudra Thakur Mr. Shivansh Verma

Guide

Dr. Aditya Turankar

Department of Computer Science and Engineering
G H Raisoni College of Engineering, Nagpur

November 2026

© G H Raisoni College of Engineering, Nagpur, 2026

Declaration

We, the undersigned students of Bachelor of Technology in Computer Science and Engineering, declare that this project report entitled VanniKawachh: A Distributed AI Acoustic Intelligence and Autonomous Drone Response Network for Women's Safety is based on the work carried out by us under the guidance of Dr. Aditya Turankar. Sources and prior work used in preparing this report are acknowledged in the text and references. Generative-AI tools were used to assist with language editing and document structure. The authors remain responsible for verifying the technical content, source citations, and reported results.

Student details and signatures

Name	**Mobile**	**Personal email**	**Signature**
Mr. Rohan Untawale	9405931361	untawalerohan@gmail.com	
Mr. Saksham Sabadra	9112904331	sabadrasaksham@gmail.com	
Mr. Rudra Thakur	9405155999	rudrasingh1108@gmail.com	
Mr. Shivansh Verma	9623688451	shivansh1411@gmail.com	

Place: Nagpur Date: November 2026

Certificate

This is to certify that the project report entitled VanniKawachh: A Distributed AI Acoustic Intelligence and Autonomous Drone Response Network for Women's Safety, submitted by Mr. Rohan Untawale, Mr. Saksham Sabadra, Mr. Rudra Thakur, and Mr. Shivansh Verma in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering, is a record of work carried out under the guidance of Dr. Aditya Turankar at G H Raisoni College of Engineering, Nagpur.

Dr. Aditya Turankar Project Guide	Prof. Atiya Khan Project In-charge
Dr. Mangala Madankar Head of Department	Dr. Sachin Untawale Director

G H Raisoni College of Engineering, Nagpur
November 2026

Acknowledgement

We express our sincere gratitude to our project guide, Dr. Aditya Turankar, for his guidance and feedback throughout this work. We thank Prof. Atiya Khan, Project In-charge, Dr. Mangala Madankar, Head of Department, and Dr. Sachin Untawale, Director, for their support. We are also grateful to the Department of Computer Science and Engineering and G H Raisoni College of Engineering, Nagpur, for the academic environment and facilities required to carry out this project. Finally, we thank our families and colleagues for their encouragement.

Abstract

VanniKawachh is a public safety system centered on an autonomous drone designed to reach reported incidents, provide an aerial view, and record visual evidence for responders. It responds only to hub verified distress alerts. At a verified location, the aircraft can clarify conditions and support a faster, better informed response. Its flight control system receives a target location and manages the journey, observation, and return phases. The drone is the response unit, while the sensing node supplies the initial alert.

The fixed sensing node listens for possible distress sounds, including screams, cries, or calls for help. It screens the audio and sends a candidate alert with its registered location to a local hub. The hub checks the available audio and context before requesting dispatch. This separation helps ensure that a single sound classification does not directly trigger the aircraft.

Together, the sensing node, verification hub, and drone form a connected response chain. The node identifies a possible event, the hub evaluates it, and the drone travels to the site to gather visual information. VanniKawachh aims to bring aerial response to incidents where conventional communication may be limited. Detection accuracy, communication reliability, and flight safety require controlled testing before operational use.

Keywords: women's safety; acoustic event detection; edge inference; audio verification; LoRa; authenticated reporting; drone response; software-in-the-loop.

Index

Declaration i

Certificate ii

Acknowledgement iii

Abstract iv

1 Introduction 1

1.1 Motivation . . . 2

1.2 The inverted-burden principle . . . 2

1.3 Problem statement . . . 3

1.4 Scope . . . 3

1.5 Contributions . . . 4

1.6 Thesis organization . . . 4

2 Literature Review 5

2.1 Survey method . . . 6

2.2 Bucket 1: victim-carried devices . . . 6

2.3 Bucket 2: detection-only audio systems . . . 6

2.4 The gap . . . 7

2.5 Technology base . . . 7

2.6 Personal safety systems: what they solve and what they assume . . . 8

2.7 Acoustic event detection and the meaning of a positive result . . . 9

2.8 Noise, distance, and temporal evidence . . . 10

2.9 Edge inference and the division of computation . . . 11

2.10 Communication and location in a distributed installation 12
2.11 Aerial response: useful capability with a higher safety burden 13
2.12 Privacy and public-space acoustic sensing 14
2.13 Comparative synthesis and research gap 15

3 Methodology and System Design **16**
3.1 Three tiers 17
3.2 Physical platform and prototype inventory 18
3.3 Current operating paths and their boundaries 19
3.4 Design decisions and rationale 21
3.5 The mission lifecycle (response tier) 22
3.6 Data flow, end to end 22

4 Data Collection and Platform **23**
4.1 Audio data, development tools, and computing platform 24
4.2 Sensing-node firmware 24
4.3 Verification hub 26
4.4 Authenticated field relay and telemetry gate 27
4.5 Alert authentication and replay protection 28
4.6 Stage-2 audio verification 29
4.7 Evidence fusion 31
4.8 Dispatch decision 32

5 Design and Implementation **33**
5.1 Hazard analysis 34
5.2 Verified mode transition 34
5.3 The landing interlock 35
5.4 Failsafe arbitration 36
5.5 Hover-phase evidence recording 37
5.6 What the software refuses to own 37

6 Testing and Summary of Results **38**
6.1 Methodology 39

6.2 Earlier flight software results . . . 39
6.3 Results: Phase-0 full-chain rehearsal . . . 40
6.4 Defect case studies (retained from v1) . . . 40
6.5 Acoustic-model and hub benchmark results . . . 41
6.6 Measurements still needed . . . 42
6.7 Safety, Privacy, and Legal Compliance . . . 43
6.7.1 Privacy by construction . . . 43
6.7.2 Alert integrity and remaining threats . . . 44
6.7.3 Flight operations and pilot control . . . 44
6.7.4 Dispatch restraint . . . 44
6.7.5 Radio-spectrum compliance . . . 44

7 Conclusion **46**
7.1 Conclusions . . . 47

8 Future Scope **48**
8.1 Future work . . . 49

A Project Photographs **54**

B LoRa Alert Packet Wire Format **56**

C Configuration Reference **57**

D Test Inventory **58**

List of Figures

3.1 Target three-tier architecture. The current bench and cloud paths have separate validation boundaries described in Table 3.2. 17

3.2 Airframe interfaces. Pixhawk owns motor commands; the companion exchanges telemetry and requests through MAVLink. 18

3.3 Target operational flow. Missing verification or failed flight-readiness checks stop automatic dispatch. 22

4.1 Two-stage screening and verification. A candidate alert does not authorize flight. 25

4.2 The sealed alert packet. The header is cleartext; the event data is encrypted and the packet is authenticated. 28

4.3 Evidence fusion and the automatic dispatch boundary. Prototype weights need calibration on measured field inputs. 31

5.1 Normal mission sequence. A fault or cancellation enters guarded recovery. 34

5.2 Earlier flight-stack interlocks. Each request passes readiness, in-flight checks, recovery, and disarm before queue reuse. 36

A.1 Project quadcopter during a supervised tethered outdoor test. 54

A.2 Project quadcopter and flight hardware before a test. 55

List of Tables

3.1 Prototype hardware, role, interface and validation status. 19

3.2 Project paths and their validation status. 20

4.1 Hub functions and their responsibilities. 27

5.1 Flight hazards and the corresponding software checks. 35

6.1 Earlier software-in-the-loop acceptance missions. 39

6.2 Stage-1 classification on 1,719 held-out clips. 41

6.3 Evidence required before claiming an integrated field system. 43

B.1 Fields in one 25-byte alert packet. 56

C.1 Main hub settings and their purpose. 57

CHAPTER 01

INTRODUCTION

1.1 Motivation

Consider where an assault actually happens. It is rarely on a well-lit arterial road under a CCTV camera. It is on the dark stretch between two streetlights, on the shortcut through a wooded campus edge, or in the far corner of a parking area at 23:40. These locations share three properties. First, they are surveillance dead zones: no camera, no patrol, often marginal or absent cellular coverage. Second, they are response dead zones: even a successful emergency call produces a ground unit minutes away through traffic. Third, and this is the property existing technology ignores, they are places where the victim is least able to operate a device: hands occupied, phone snatched, discharged, or simply out of reach in the seconds that matter.

The market's answer has been to instrument the victim. Panic-button mobile apps, smart wearables, GPS pendants, and IoT panic devices all place the trigger on the person at risk. The best of these report high accuracies, but the failure mode is structural: a safety device the victim must carry, charge, and reach protects only the moments in which she can carry, charge, and reach it. The research community's alternative, audio surveillance systems that detect screams, has demonstrated strong detection rates, but on server-scale compute, and it stops at detection. The output is a classifier score, not a located alert or a response on the ground.

What is missing is infrastructure that owns the whole chain: sensing that asks nothing of the victim, verification that suppresses false alarms well enough to act on, alert delivery that works precisely where cellular does not, and a first response that arrives in seconds rather than minutes. VanniKawachh is a design, an implementation, and a staged validation of that chain.

1.2 The inverted-burden principle

VanniKawachh's single organizing idea is that the burden of triggering help must move from the victim to the environment. A scream, a cry, or a shouted "help" / "bachao" is a signal the victim produces anyway, under any circumstance, with no device. If pole-mounted infrastructure can hear it, verify it, and act on it, then the victim's participation requirement drops to zero: no app, no wearable, no phone, no button.

The engineering consequences of that idea drive every design decision in this thesis:

- **Nodes must be cheap, solar, and everywhere.** So the node is an ESP32-S3 class microcontroller with a TinyML Stage-1 model rather than a streaming link to a server.

- **False alarms must be suppressed before anything expensive happens.** A drone launch is costly and attention-consuming, so a second, heavier verification stage runs at a hub, fused with environmental evidence.
- **The alert path must not depend on cellular coverage.** The target locations are dead zones by definition, so the alert travels over LoRa.
- **An alert that launches a drone must be unforgeable.** So every packet is encrypted, authenticated, and replay-protected.
- **The response aircraft must be trusted to fly unattended.** So the response layer is a flight stack whose safety properties are pinned by automated tests, built and validated before this concept existed (v1 of this project) and carried over unchanged.

1.3 Problem statement

Design and implement a distributed safety network in which pole-mounted acoustic nodes detect distress audio on-device, a locality hub confirms each event with deep audio analysis fused with environmental evidence, confirmed alerts travel encrypted over LoRa with the node's surveyed coordinates to a police-facing dashboard, and an autonomous quadcopter is dispatched to the incident to record evidence during a guarded observation window, such that (i) no continuous audio leaves any node; (ii) no alert can be spoofed or replayed into a drone launch; (iii) no dispatch occurs below explicit verification and severity thresholds; (iv) every flight-mode command is confirmed from autopilot telemetry, no mission can start against an airborne vehicle, and hazard responses obey debounce, severity, and escalation semantics; and (v) each of these properties is demonstrated by automated, reproducible tests before it is claimed.

1.4 Scope

In scope: the three-tier architecture; the node firmware structure and its Stage-1 pipeline; the complete hub service (packet cryptography, registry, Stage-2 verification, evidence fusion, dispatch gating); the response flight stack and its evidence-recording extension; controlled Stage-1 and hub-model benchmarks; end-to-end validation in software-in-the-loop simulation; and the physical hardware inventory and field-test gates. Out of scope for the prototype, and stated as such wherever relevant: hardware range, latency, endurance and

field-false-alarm measurements; live video streaming to police, multi-node time-difference localization, and beyond-visual-line-of-sight flight operations.

1.5 Contributions

1. An **end-to-end architecture** (infrastructure sensing → two-stage verification → offline encrypted alerting → autonomous field response) where prior work covers at most one link of the chain (Chapters 2 and 3).

2. A **two-stage edge/hub acoustic pipeline**: a recall-tuned MFCC + CNN screen on the node (<50 ms budget) and a precision-tuned PANNs + sensor-fusion confirmation on the hub, with explicit thresholds gating dispatch (Chapter 4).

3. A **secure LoRa alert protocol**: a 25-byte sealed packet (AES-128-CTR + truncated HMAC-SHA256 + per-node monotonic replay counter, per-node keys derived from a master key) carrying an event class rather than coordinates, resolved against a surveyed node registry at the hub (Chapter 4, Appendix B).

4. A **safety-verified autonomous response layer** inherited from the group's v1 flight stack (verified mode transition, landing interlock, failsafe arbitration), extended with an evidence-recording window during hover (Chapter 5).

5. A **staged validation record**: an earlier 68-case software-test inventory, an 8/8 SITL acceptance flight, and a zero-hardware rehearsal (Phase 0). These are historical software results with heuristic stand-ins labelled as such (Chapter 6).

1.6 Thesis organization

Chapter 2 surveys the literature and locates the gap. Chapter 3 presents the three-tier design and its rationale. Chapter 4 details the sensing node and hub implementation. Chapter 5 details the response layer: the safety engineering of the flight stack, retained from v1 because it is unchanged and still load-bearing. Chapter 6 defines the validation methodology and reports results, including what has not yet been measured. Chapter 6.7 consolidates the safety, privacy, and legal posture. Chapter 7 concludes.

CHAPTER 02

LITERATURE REVIEW

2.1 Survey method

The group's literature survey (twelve papers, 2023 to 2026, first catalogued in the Title Finalization Seminar record and reproduced with full bibliographic detail as references [1] to [12] of this thesis) covered the women-safety technology space across two communities: consumer/IoT safety devices and acoustic event detection. Every surveyed system falls into one of two buckets, and the buckets split on a single question: who carries the trigger?

2.2 Bucket 1: victim-carried devices

Mobile panic applications, smart wearables (bands, pendants, footwear sensors), and standalone IoT panic buttons [1, 5, 6, 9, 10, 11] place the sensing and triggering hardware on the victim. Pavithra et al. [1] present a discreet smartphone application with on-device audio recognition and emergency automation; Bharathi et al. [5] describe an IoT wearable combining GPS and accelerometer sensing and report **97.54%** detection accuracy with a 3.2 s response time and a 1.92% false-positive rate; Snehith et al. [6] and Hadkar et al. [9] present push-button/wearable IoT devices with locate-and-alert behaviour; Uganya et al. [10] describe a button-triggered GSM/GPS tracker; and Potturi et al. [11] automate the SOS trigger with GSM/GPS geo-location sharing. The stronger systems combine multiple modalities (accelerometer gestures, heart-rate anomalies, voice keywords), and GPS positioning with GSM/app-based alerting is near-universal in this bucket.

Availability remains an important issue alongside classifier accuracy. A carried-device trigger may be inaccessible when the device is absent, damaged, discharged, or out of reach. Its reported accuracy does not measure this access failure. App and GSM alerting also depend on coverage, so each proposed site needs a network assessment [10].

2.3 Bucket 2: detection-only audio systems

The acoustic event detection literature [2, 3, 4, 7, 8] demonstrates that distress audio is machine-detectable at useful rates. Kim et al. [7] built an 11,921-sample large-scale scream dataset and found a CNN-Transformer to be the best of five evaluated models; the same group [2] later added a windowing CNN for scream temporal-interval prediction, improving F-measure by roughly three percentage points and equal-error rate by an order of magnitude. Sharma and Jebaseeli [3] report a scream-and-panic detector at **92%** accuracy with under 5% false positives; Fime et al. [8] achieve **95.51%** danger-sound accuracy with

Noisereduce preprocessing and transfer-learned image backbones (InceptionV3, with MobileNetV2 as a lighter alternative) over mel-spectrogram inputs; and Srimathi et al. [4] show that scream detection can reduce reaction time by 40% and increase intervention rates by 30%. Ciaburro and Puyana-Romero's systematic review of sound-event detection in smart cities [12] confirms both the maturity of detection methods and the near-absence of closed-loop response systems in the literature.

Three limitations matter for this project. First, the compute and power budget of a fixed solar node differ from those in larger-model experiments [2, 7]. Second, results from curated recordings do not predict an outdoor microphone's false alarms. Third, many studies evaluate classification without also measuring authenticated location reporting and a guarded physical response.

2.4 The gap

Overlaying the buckets exposes the gap this project fills. Bucket 1 has response paths (alerts to guardians/police) but burdens the victim. Bucket 2 removes the victim's burden but has no response path. **No surveyed system provides end-to-end infrastructure sensing $\rightarrow$ verified offline alerting $\rightarrow$ autonomous field response.** That chain, with each link individually feasible on published evidence, is VanniKawachh's contribution, and each link imports a discipline from a different literature: TinyML keyword spotting for the node [16], large-scale pretrained audio tagging (PANNs [13]) for the hub, LPWAN practice for the alert path [18, 34], and the group's own SITL-verified dispatch stack for the response [17, 24, 25, 26].

2.5 Technology base

TinyML on microcontrollers. TensorFlow Lite Micro [16] supports quantized models on microcontrollers. Its example pipelines inform the proposed MFCC-fronted node model, but target-board RAM use and timing must be measured on the actual ESP32-S3 configuration.

PANNs. Pretrained Audio Neural Networks [13] (CNN14 and lighter variants trained on AudioSet) produce class scores over hundreds of sound events, including the distress-relevant family (screaming, shouting, crying, yelling, wailing). Combining relevant classes yields a candidate distress score, but its threshold still needs evaluation on target audio. This makes PANNs an option for Stage-2 verifier on a Raspberry Pi 5.

LoRa. Semtech SX1278-class radios [34] deliver low-rate links with no cellular operator (the LoRaWAN specification [18] documents regional parameters). Actual range and throughput depend on frequency, settings, and site. The nominal data rate is enough enough for a 25-byte alert and far too little for audio, which shapes the split transport design of Section 3.4.

The autopilot stack. ArduPilot and MAVLink [17] and software-in-the-loop simulation [26] are covered in the group's v1 work; Chapter 5 summarizes what carries over. The v1 finding that motivates the whole project's verification philosophy bears repeating: the standard client idiom for changing flight mode was observed to fail silently on ArduCopter 3.3 SITL: the library reports success while the autopilot ignores the command. The design answer, a detection claimed is not a detection confirmed; a command sent is not a command adopted, now governs every layer of VanniKawachh: sound, dispatch, and flight mode alike.

Regulation. India's Drone Rules, 2021 (G.S.R. 589(E)) [21] govern the prototype's flight operations (registration, zone constraints, visual line of sight); the Digital Personal Data Protection Act, 2023 [22] and plain privacy prudence govern the audio design; and the Wireless Planning and Coordination Wing's delicensing framework [23] governs the radio links. Chapter 6.7 consolidates all three.

2.6 Personal safety systems: what they solve and what they assume

Personal safety applications address a practical need: a person should be able to contact trusted people or emergency services without having to remember a long sequence of steps. The systems surveyed in this work approach that goal through a phone application, a wearable device, or a dedicated button. Their implementations differ, but the common workflow is recognizable: the user activates a trigger, the device packages an alert and some location information, and a network path carries the message to a contact or service. Pavithra et al. explore audio recognition and automated actions within a phone application, while Potturi et al. describe automatic alerts and location sharing using GPS and GSM [1, 11]. These systems reduce the number of manual operations required after activation.

They do not, however, remove the dependency on the phone being present, charged, accessible, and connected. Wearable approaches add sensors that can recognize a gesture or infer a possible emergency. Bharathi et al. discuss a design combining location, movement, audio, and physiological sensing; Snehith et al. and Hadkar et al. describe IoT safety

devices; and Uganya et al. present a GPS/GSM tracker with a manual trigger [5, 6, 9, 10]. Each arrangement reflects a reasonable product choice: sensing on the person can follow the user, and a personal device can be configured with the user's contacts. It also creates a maintenance obligation. The user must remember to carry, charge, and keep the device functional. A user may also be unable to operate it during a rapidly developing event.

This is not a flaw unique to one paper; it is a design boundary shared by systems whose alarm begins on a personal device. Reported performance values in these papers should be read in the context of their specific datasets, sensors, test conditions, and definitions of an emergency. A classifier's accuracy does not measure whether a user is wearing the device, whether GPS has a useful fix, or whether an alert reaches a responder. Similarly, a short measured alert delay in a controlled prototype does not establish a guaranteed response time in a different network or city. For the present project, the relevant lesson is therefore not that a fixed public node will always outperform a wearable.

Rather, the two architectures cover different conditions. A carried device follows its owner. A fixed node may help at a known location without requiring the person to activate equipment. VanniKawachh investigates the latter setting and keeps personal devices outside the required trigger path.

2.7 Acoustic event detection and the meaning of a positive result

Sound-event recognition offers a way to detect a possible emergency without waiting for a button press. Its appeal is also its main challenge: many ordinary events can resemble distress in a short recording. A loud vehicle, children playing, a sports call, a musical sound, or a clipped microphone signal may all produce a strong classifier response. A system must therefore be evaluated not only on whether it recognizes selected positive examples, but also on how it behaves across long periods of ordinary background audio. The studies by Kim et al. [2] develop scream recognition with attention to noisy conditions and temporal localization. Their later CNN–Transformer and Windowing CNN system models local spectral patterns and temporal context, and predicts the interval associated with a scream rather than returning only a clip-level decision.

This is relevant to an always-listening node because the location in time of an event can guide which short segment should be preserved for review. The earlier large-scale scream-dataset work provides a complementary dataset and model-evaluation perspective [7]. These contributions show why a distress system needs to consider both classifica-

tion and the event's temporal boundaries. Sharma and Jebaseeli examine scream and panic recognition as a safety application, while Srimathi et al. consider how detection can be connected to an alerting workflow [3, 4]. Fime et al. study danger-sound classification with preprocessing and deep-learning methods, including approaches designed for noisy recordings [8]. Across these papers, the choice of label set, recording source, train/test partition, and noise construction affects the meaning of the resulting score.

A model trained to distinguish a particular scream dataset from selected negatives is not thereby validated for every street, microphone, language, distance, or acoustic reverberation. This distinction matters for VanniKawachh. The system's first classifier is a screening stage: a positive result means that a short window should be passed to a second decision, not that an incident has been proven. The hub's Stage-2 score and contextual inputs are separate evidence, and a policy gate must still be met before a mission request is created. Conversely, a negative screening result can miss a quiet or distant event, so the screening model's recall and microphone coverage remain important field questions. The thesis reports the current held-out dataset metrics as model measurements and does not reinterpret them as an outdoor false-alarm rate.

2.8 Noise, distance, and temporal evidence

Real outdoor sound differs from a clean training clip in several ways at once. The microphone may be far from the speaker; a wall or vehicle may obstruct the path; the source may move; and wind, traffic, crowd sound, or electrical noise may mask parts of the signal. Reverberation can stretch a short vocal event, while automatic gain control or clipping can change its amplitude profile. These effects can change the features supplied to a model even when the person produces the same word or scream. For that reason, a single spectrum or fixed frequency threshold is not a dependable distress rule. The noisy-condition experiments in Kim et al. are useful because they treat noise as part of the evaluation setup and consider the localization of the scream interval [2].

Fime et al. likewise examine preprocessing and classification in danger-sound recognition [8]. These studies support evaluation across distinct acoustic conditions, but their reported results remain tied to their own mixtures, sensors, and test protocols. A noise type represented in a dataset is not equivalent to every field combination of distance, microphone placement, weather, and background activity. Temporal evidence can help distinguish a sustained background from a short human event, but it too is context-dependent. The Stage-1 front end in this project frames audio and computes MFCC features so the compact model can compare changes across successive windows. The hub can examine a longer event clip

and associate its score with the initial screening confidence. The design therefore uses the waveform as a sequence, not as a single instantaneous loudness value.

Motion, illumination, and time-of-day provide additional context, but none of these measurements proves distress on its own. A PIR sensor may respond to benign movement; a low light reading may simply describe night-time conditions; and the hour is only a weak prior. The fusion stage is intended to combine these imperfect cues while retaining an independent acoustic threshold. The literature suggests two practical evaluation layers. Laboratory tests should vary signal-to-noise ratio, source distance, and event duration under repeatable conditions. Field tests should then record representative background periods and report false alarms per unit of monitoring time, missed events, end-to-end latency, and operating range. The second layer is necessary because clip-level classification metrics alone do not capture how often a continuously listening installation would raise an alert in daily use.

VanniKawachh has not yet completed that field campaign; it is listed as future validation, not inferred from the held-out clip score.

2.9 Edge inference and the division of computation

An infrastructure microphone could send all audio to a central computer, but continuous streaming increases bandwidth use, power demand, network dependence, and privacy exposure. A small model at the microphone offers a different arrangement: analyse short frames locally and transmit only when a candidate event is found. TensorFlow Lite Micro provides an implementation path for running quantized neural models on microcontrollers with constrained memory [16]. The review by Ciaburro and Puyana-Romero surveys sound-event detection in smart cities and the trade-offs involved in processing acoustic data [12]. The trade-off is between model capacity and operational cost. A small quantized model can run close to the microphone, but it may have less representational capacity than a larger model.

A central model can examine more context, but it requires an available link and enough processing resources. The VanniKawachh architecture assigns the low-cost task to the ESP32-S3 and the more detailed review to the hub. This division also supports a useful safety boundary: the edge device can request review but cannot authorize the aircraft. Should a model update, feature-extraction change, or runtime problem occur, the effect can be isolated and tested at the stage where it belongs. The same division affects measurement. Model size is only one resource metric. A field-ready edge implementation also needs its

inference latency, peak memory, power draw, thermal behavior, and watchdog/recovery behavior measured on the exact board and firmware build.

A desktop benchmark cannot substitute for those measurements. Likewise, a model that fits in flash may still be too slow for the intended frame cadence. The current thesis reports the deployed model's size and the separate held-out classification result, while explicitly listing board-level energy and timing measurements as incomplete. Pretrained audio models provide another way to allocate computation. PANNs use largescale audio pretraining and include CNN-based architectures that produce scores for multiple sound classes [13]. Such a model can provide a second opinion using a broader vocabulary than a tiny screening network. It does not eliminate the need to validate the target labels, calibrate scores, or test the specific Raspberry Pi runtime. A high distress-related class score can still occur in a non-emergency sound, so the hub's role is verification under a policy, not unquestioned acceptance of a pretrained model's output.

2.10 Communication and location in a distributed installation

Emergency reporting has two different communication needs. The system must send a small, urgent message with the identity of the reporting device and enough information to retrieve the event. It may also need a larger audio segment for verification. A single low-rate radio link is poorly suited to both tasks. LoRa has been investigated for emergency-management situations where ordinary mobile infrastructure may be absent [15, 18]. Its low data rate is compatible with a compact alert, but makes it a poor choice for transferring an audio clip. This motivates the split path in VanniKawachh: a short LoRa packet reports the event and a separate local wireless link carries the bounded clip when available. The separation creates its own failure modes.

The LoRa alert may arrive while the audio link is unavailable, or the clip may arrive after the verification window has expired. The receiver must associate both messages with the same event, reject stale or mismatched clips, and record when the verification evidence was absent. Treating a missing clip as if it were a successful Stage-2 classification would hide uncertainty. The system instead records the alert with Stage-2 confirmation unavailable and does not dispatch. Radio range, packet delivery rate, interference, antenna orientation, and latency must be measured at the proposed installation site. Protocol documentation alone does not establish coverage for a particular pole or neighborhood. Location is also a system property rather than merely a GPS reading.

A fixed sensor can be surveyed at installation and registered against a node identifier. This avoids requiring a fresh coordinate fix for every alert, but makes correct installation records and registry integrity important. The node identifier must map to the correct installed coordinates, and a maintenance process is needed if hardware is moved or replaced. A mobile GPS/GSM tracker and a fixed node registry therefore have different error sources. VanniKawachh's design uses the surveyed location as the incident origin and treats GPS availability on the aircraft as a separate flight readiness condition. Security must cover message authenticity and freshness as well as radio delivery. Encryption without authentication does not prove that an event came from a registered node, and a previously valid alert should not be accepted repeatedly as a new incident.

The packet design therefore includes an authentication tag and a monotonic event counter, with the gateway kept as a forwarder rather than a dispatch authority. AES and HMAC are standard building blocks for confidentiality and integrity, respectively [19, 20]. Implementation choices such as key provisioning, counter persistence, tag length, and recovery after power loss still require careful treatment. Chapter 4 gives the implemented packet format and the specific limitations of its compact tag.

2.11 Aerial response: useful capability with a higher safety burden

An aerial vehicle can reach a location without following the same road path as a ground responder, and its camera can record conditions at the incident site. These properties have motivated research into drone-assisted emergency response. For the present thesis, the important point is not to assume that an aircraft can replace emergency services. A drone has a limited battery, weather and airspace constraints, and its own risks around people and buildings. It can only be considered as a supplementary response platform whose mission is bounded and whose operator retains a safe way to intervene. The control chain creates a different class of failure from acoustic classification. A correctly detected event can still lead to an unsafe outcome if a target coordinate is wrong, an autopilot mode change is not accepted, the vehicle has a stale position estimate, the battery is insufficient, or a previous mission has not reached a safe terminal state.

This means a mission command must be confirmed through telemetry rather than assumed successful because an API call returned. ArduPilot, MAVLink, and SITL provide the software interfaces used to test these state transitions [17, 26]. The mission queue, target checks, geofence, landing lock, and ordered failsafe actions in this project address these

software-side hazards. They do not prove the physical aircraft's behavior in wind, vibration, GPS multipath, or component failure. The literature on personal safety and acoustic recognition generally evaluates sensing or alerting separately from an autonomous aircraft. This project connects the two, which increases the need for conservative authorization. An acoustic candidate cannot directly trigger motors or change flight mode. First, packet authenticity and freshness are checked.

The hub then evaluates the available sound and context. Finally, the flight service checks its own readiness conditions and waits for autopilot confirmation. Each layer can refuse the next transition. This design does not make the system fail-proof, but it makes the basis for a dispatch auditable and prevents one classifier output from becoming a flight command. Evidence recording raises a distinction between a software rehearsal and a validated field capability. A successful SITL mission confirms that the recording window and return sequence can be exercised in simulation, but it does not establish camera coverage, image quality, or performance in outdoor lighting and weather. Those measurements require supervised physical tests. This boundary is important in a safety project: a completed software command sequence is not, by itself, proof of effective evidence capture near the public.

2.12 Privacy and public-space acoustic sensing

Fixed microphones create a privacy question even when the system is intended for emergencies. A system that stores or streams all ambient audio could collect ordinary conversations unrelated to safety. On-device processing, short event-triggered clips, limited retention, and access controls reduce that exposure, but they do not make privacy concerns disappear. Public deployment would also require clear notice, a defined purpose, a lawful basis, a retention and deletion schedule, and an accountable operator. The Digital Personal Data Protection Act, 2023 is part of the Indian legal context considered in this thesis [22]. The design's privacy objective is data minimization: the node processes ordinary frames locally and does not intentionally store or stream them. A short verification clip may be created after a candidate event and should have a documented retention period.

Alert logs should preserve enough metadata to audit system behavior without turning into a permanent record of ambient sound. Access to recordings and incident details should be limited to the personnel responsible for response and review. The prototype's software configuration and a future production policy must be distinguished. the latter still needs formal review before deployment. Privacy design also interacts with evaluation. Researchers need enough recorded data to measure false alarms and missed events, yet collecting continuous

audio solely for convenience conflicts with the minimization objective. A field study can instead define a reviewed collection protocol, obtain the necessary institutional approvals, restrict access, and retain only the samples required for analysis. Data handling must be designed before recording begins rather than added after the system is already in operation.

This makes privacy a system requirement alongside latency, security, and flight safety, not a closing paragraph appended after the hardware design.

2.13 Comparative synthesis and research gap

The reviewed work supplies evidence for separate parts of an emergency response chain. Personal applications and wearables show how a user can initiate an alert and share a location, but they depend on a device carried by the person [1, 5, 6, 9, 10, 11]. Acoustic research shows that screams and other danger sounds can be classified, and that temporal models can locate a scream interval in a recording [2, 7, 3, 4, 8]. Edge-inference work supports moving part of the classification closer to the microphone [16, 12]; emergency-communications research supports low-rate radio where cellular access is unreliable [15, 18], and flight-control tooling makes it possible to exercise mission logic in simulation [17, 26]. These findings do not imply that the parts are interchangeable or that their published performance transfers directly to a new deployment.

Each paper uses a different dataset, hardware platform, radio configuration, mission policy, and success measure. A classifier's benchmark does not measure radio reliability, an alerting prototype does not establish aircraft safety, and a simulated mission does not measure real endurance. The value of the survey is to identify which design decisions have precedent and which claims still need project-specific testing. The gap addressed here is therefore an integration and verification gap. VanniKawachh places the initial microphone at a fixed location, screens sound locally, sends a compact authenticated alert separately from the audio, verifies the event at a hub, and only then requests a drone mission through a state-checked control service. The project contribution is this connected workflow and its software-level evaluation.

It is not a claim that the prototype has already established public-space detection performance or safe autonomous operation in the field. The following chapters describe the architecture and implementation in sufficient detail to separate what has been built and tested from what remains to be measured.

CHAPTER 03

METHODOLOGY AND SYSTEM DESIGN

3.1 Three tiers

Figure 3.1 summarizes the target three-tier workflow and the distinct alert, audio, and telemetry paths. Each tier has a bounded role: the node screens, the hub verifies and authorizes, and the flight controller reports the vehicle state used by the mission executor.

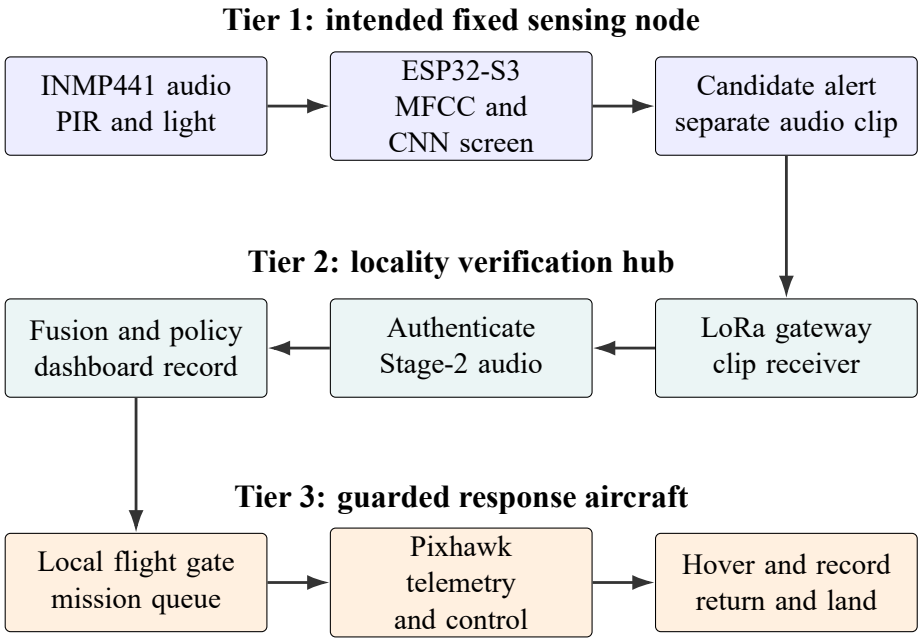


Figure 3.1: Target three-tier architecture. The current bench and cloud paths have separate validation boundaries described in Table 3.2.

Tier 1: sensing node. One per pole. An ESP32-S3 continuously frames 16 kHz mono audio from an INMP441 I2S microphone, extracts MFCC features, and runs a quantized CNN with TensorFlow Lite Micro against the distress vocabulary: scream, cry, and the "help" / "bachao" keyword family. Budget: under 50 ms per frame. This is a design budget, not a measured result. An HC-SR501 PIR sensor and an LDR would provide motion and ambient-light context sampled alongside. A Stage-1 hit would send a sealed 25-byte alert over LoRa and a short audio clip over a separate faster link. The complete pole hardware path has not been validated outdoors.

Tier 2: hub. One Raspberry Pi 5 per locality. A gateway ESP32 with an SX1278 bridges LoRa to USB serial. The hub authenticates and decrypts each packet, resolves the node's surveyed coordinates from a registry, waits briefly for the WiFi clip, re-scores the audio with an available learned backend, then computes fused severity using measured context where available. Automatic dispatch requires Stage-2 confirmation and a severity gate. Software records an incident whether or not it requests a mission; this is not a claim that a deployed police dashboard received an alarm.

Tier 3: response drone. The earlier flight software provides a FastAPI [27] request interface, a priority mission queue, and a mission state machine with failsafe checks. The current design adds a camera recording window during hover. Simulation results for the earlier flight software do not establish operation through the newer cloud bridges.

3.2 Physical platform and prototype inventory

The software architecture is meaningful only when its physical interfaces are stated clearly. Figure 3.2 shows the response airframe at the electrical-interface level. The LiPo battery feeds a power module, which provides voltage/current telemetry to Pixhawk; the distribution path supplies the four ESCs, which in turn drive the motors and propellers. Pixhawk receives GPS, compass and RC receiver inputs, and sends PWM commands through MAIN OUT 1–4 to the ESCs. The companion/relay link uses MAVLink through TELEM2 rather than attempting to command motors directly. Thus, the flight controller retains ownership of stabilisation, EKF navigation, arming checks and onboard failsafes.

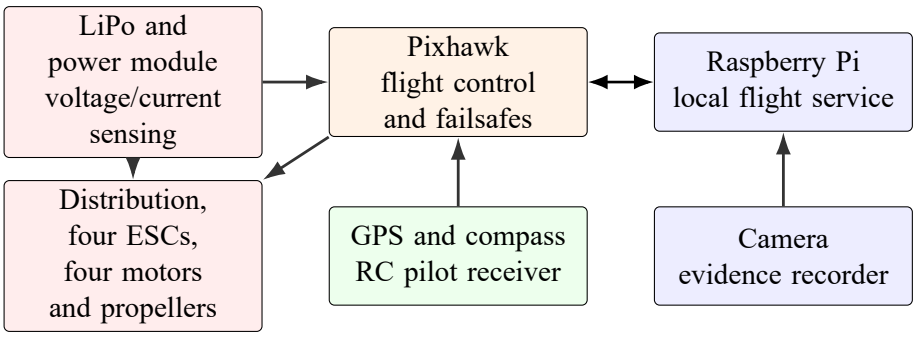


Figure 3.2: Airframe interfaces. Pixhawk owns motor commands; the companion exchanges telemetry and requests through MAVLink.

Table 3.1 lists target and bench components with their validation status. A listed component is not necessarily part of a completed sensing-node or flight demonstration.

Table 3.1: Prototype hardware, role, interface and validation status.

Assembly	Component and role	Principal interface	Status in this thesis
Sensing node	ESP32-S3, INMP441 microphone, PIR and LDR	I2S, GPIO/ADC	Offline model and firmware; complete target node unverified
Sensing radio	SX1278 LoRa radio with 433 MHz antenna	SPI/LoRa	Packet path implemented; range pending
Node location	NEO-6M for installation survey	UART	Registry design; pole installation not documented
Hub	Raspberry Pi 5, USB LoRa gateway ESP32, local services	USB serial, WiFi/Ethernet	Software and relay integration
Airframe	F450 frame, landing gear and 1045 propellers	Mechanical	Physical prototype platform
Propulsion	4×2212 920 kV BLDC motors and 4×30 A ESCs	PWM MAIN OUT 1–4	Physical propulsion subsystem
Flight control	Pixhawk 2.4.8 running ArduPilot	GPS, RCIN, PWM, TELEM2	Hardware platform; SITL and read-only bench evidence
Navigation	u-blox M8N GPS plus compass	GPS/UART and I^2C	Required for guided-flight readiness
Power	3S/4S LiPo, power module, distribution board and 5 V BEC	Power, voltage/current telemetry	Battery telemetry calibration pending
Pilot safety	FlySky FS-i6 receiver/transmitter and manual override	RCIN	Required for every physical test
Companion and camera	Pi-class companion with camera mount	MAVLink and camera interface	Recording software implemented; field validation pending

3.3 Current operating paths and their boundaries

Figure 3.1 is the target three-tier arrangement, not a claim that a pole network and autonomous drone are deployed. Table 3.2 distinguishes the physical and software paths present in the repository. This distinction matters because they use different microphones, communications links, verification engines, and flight gates.

Table 3.2: Project paths and their validation status.

Path	Available project artifacts	Validation status
Intended fixed node	ESP32-S3 and INMP441 design, offline classifier, sensor and radio interfaces.	Full node integration, board timing, radio range and pole installation remain unverified.
KY-037 bench node	ESP32-S3 sends signed sound-level candidates to a Pi; a newer sketch adds a clip.	Quiet-condition false alerts observed. Clip sketch still needs upload and hardware validation.
Phone and USB microphone	Browser and Pi microphone routes send clips for cloud verification.	These routes lack PIR and light measurements; a cloud queue does not prove flight.
Drone-side Pi	Flight API and cloud bridge check telemetry, home, GPS, battery and pilot state.	Physical dispatch is locked; bench GPS and battery data were unavailable.
Drone-side ESP	Bridge reads the cloud queue and can upload a Pixhawk mission.	Build and software tests only; no documented arming or flight.
Earlier flight stack	Mission state machine, interlocks and failsafe checks.	Eight simulation acceptance checks passed; newer bridges remain unverified.

The phone route documented for automatic queuing requires authenticated reporting, a verified audio incident, an allowed verification backend, and live location accuracy of 20 m or better. A separate operator request creates a short-lived real-mode test command. That command bypasses audio verification for a supervised test, but it does not bypass the Pi's local readiness checks, the flight API, or Pixhawk pre-arm behavior. In the recorded bench configuration, real dispatch is locked. The cloud stores incident and command state on an ephemeral SQLite filesystem, so it cannot yet be described as a dependable emergency transport.

The KY-037 route is not equivalent to the intended INMP441 and CNN route. The former observes sound level and has produced false candidates; the latter's 86.62% result is an offline dataset metric. The newer KY-037 sketch transfers a short clip for Pi-side verification only after upload and hardware checks. Neither result establishes field precision. Phone and USB-microphone paths currently supply default PIR and light values, so those context terms are not sensor measurements on these routes.

For physical flight, the Pi and ESP bridges are alternatives. The Pi route checks fresh telemetry and a current launch home before asking the local API to dispatch. The ESP bridge's mission upload does not arm or start flight. The response photos document airframe work, not an autonomous node-to-drone demonstration.

3.4 Design decisions and rationale

Fixed nodes carry no live GPS. Each pole is surveyed at installation using a GPS receiver or phone. The hub stores the node identifier, coordinates and location name. The alert needs only two bytes for the identifier instead of carrying a coordinate pair. This reduces packet size. The hub rejects an identifier that is absent from its registry.

LoRa carries the alert; WiFi carries the audio. A compact alert is suited to LoRa's low data rate. A four-second verification clip needs a faster local link, so the intended design sends the alert over LoRa and the clip over WiFi or ESP-NOW. Range, delivery rate and clip latency still need measurement at an installation site.

The split is a direct consequence of LoRa's physical layer. The duration of one LoRa symbol at spreading factor SF over bandwidth BW is

$$T_s = \frac{2^{\mathrm{SF}}}{\mathrm{BW}} \tag{3.1}$$

so higher spreading factors buy sensitivity at the price of airtime. The receiver's minimum detectable signal is

$$P_{\mathrm{min}}\,[\mathrm{dBm}] = -174 + 10\log_{10}(\mathrm{BW}) + \mathrm{NF} + \mathrm{SNR}_{\mathrm{min}} \tag{3.2}$$

where NF is the receiver noise figure and $\mathrm{SNR}_{\mathrm{min}}$ the demodulation threshold for the chosen SF. The proposed 25-byte alert needs much less airtime than a four-second, 16 kHz, 16-bit audio clip of roughly 128 kB. Sending the clip over the same low-rate radio would introduce a large delay. This is the reason for separate alert and audio links; it does not establish actual radio coverage or clip delivery time.

If the clip does not arrive within 8 s, the hub logs the candidate without a dispatch request. A Stage-1 score alone cannot replace independent audio verification. Multi-node corroboration slots into the same fallback point as future work.

Screen first, verify before dispatch. The node sends a candidate for review; the hub independently checks the audio and available context. Stage 1 can miss events, as its measured scream recall shows, and Stage 2 can also make errors. The two-stage design limits direct dispatch from one classifier score while leaving both false alarms and missed events for field evaluation.

Authenticate each alert. A forged packet must not enter the dispatch chain as a trusted event. Section 4.5 describes the packet encryption, authentication and replay check.

Flight checks remain separate. The earlier flight software provides confirmed mode

setting, failsafe handling, landing interlocks, geofence checks and stalled-route detection. These checks form a software baseline; Chapter 5 describes their scope and the evidence still needed for physical operation.

3.5 The mission lifecycle (response tier)

The response sequence runs from IDLE through CONNECTING, WAITING GPS, ARMING, TAKEOFF, ENROUTE, and HOVERING, then returns through RTL and LANDED to COMPLETED. ABORTED and FAILED are abnormal terminal outcomes. Three earlier properties are preserved: no state waits for human input; every transition is timestamped into the mission log; every blocking loop polls both the failsafe arbiter and the operator-cancel flag, so abnormal termination is reachable from the active flight phases. The hover window is the intended evidence-recording interval.

3.6 Data flow, end to end

Figure 3.3 shows the intended automatic route from a candidate sound to a guarded response. This is a software design target, not a completed physical integration.

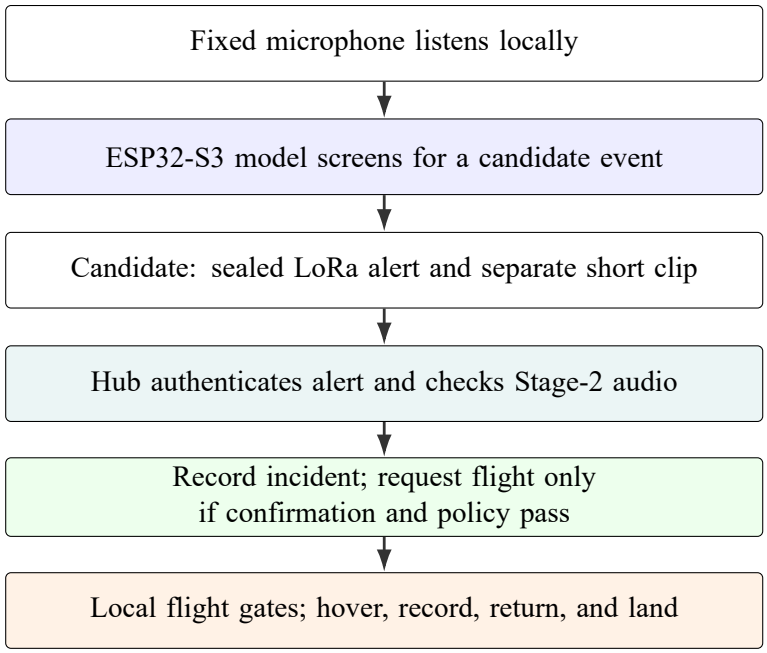


Figure 3.3: Target operational flow. Missing verification or failed flight-readiness checks stop automatic dispatch.

CHAPTER 04

DATA COLLECTION AND PLATFORM

4.1 Audio data, development tools, and computing platform

The Stage-1 training record groups background, scream, cry, and emergency-word clips into a four-class dataset. Audio is converted to a consistent sample rate and feature shape before training. Its reported evaluation contains 1,719 windows, but the split is by source file rather than by speaker. The checked-in metrics do not include a complete source manifest, command-line invocation, and hardware run log. This limits reproducibility and can make results optimistic when nearby files share a speaker or acoustic setting. The report therefore gives both the overall 86.62% result and the weaker class result: scream recall is 55.52%, while background false positives are 7.43% in that evaluation.

A separate hub-side feature classifier uses a YAMNet TFLite representation and a speaker-disjoint ASVP-ESD holdout. Its 218-example internal result is 97.25% accuracy with four false positives and two false negatives. ASVP-ESD also contributed training examples, so this is not a source-external test. These two evaluations answer different questions and cannot be averaged or presented as a single system accuracy. The run records do not measure live outdoor precision, inference latency on the sensing board, or the false-alarm rate of the KY-037 bench route.

Development uses Python tools for dataset preparation, training, hub services, and ArduPilot simulation. The intended node executes on an ESP32-S3; the locality hub targets a Raspberry Pi 5; Pixhawk owns flight stabilization and motor control. The phone and Pi USB microphone are current alternate audio inputs, not evidence that the intended pole node is running its model in the field.

4.2 Sensing-node firmware

The sensing node is a single Arduino sketch for the ESP32-S3 [33] structured as the pipeline of Section 3.1: I2S capture from the INMP441 [35] at 16 kHz mono; framing and MFCC feature extraction; the TFLite-Micro Stage-1 model hook; PIR and LDR sampling; AES-sealed LoRa alert transmission (SX1278, LoRa library by Sandeep Mistry); and the ESP-NOW/WiFi clip upload to the hub's clip server. The alert carries the event class (1 = scream, 2 = call for help, 3 = cry, 4 = crash), the Stage-1 confidence quantized to a byte, the PIR flag, the raw LDR level, the node battery percentage, and the monotonic packet counter. Figure 4.1 shows the planned two-stage pipeline. The offline Stage-1 model export and firmware interface do not by themselves establish measured on-board inference.

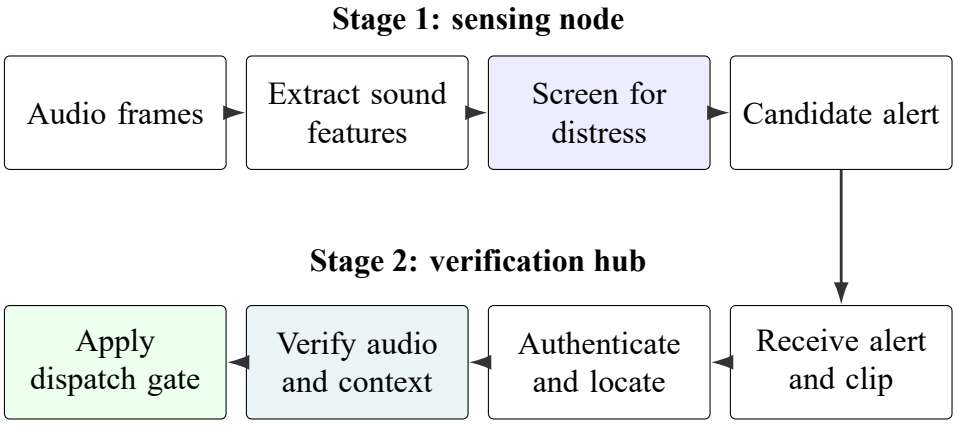


Figure 4.1: Two-stage screening and verification. A candidate alert does not authorize flight.

The Stage-1 mathematics. The feature front-end is the classical MFCC chain. Each incoming frame is first pre-emphasized to flatten the spectral tilt of speech,

$$y[n] = x[n] - 0.97\, x[n-1] \tag{4.1}$$

then windowed with a Hamming window of length N,

$$w[n] = 0.54 - 0.46 \cos\!\left(\frac{2\pi n}{N-1}\right) \tag{4.2}$$

before the FFT. The power spectrum is pooled by a bank of M triangular filters spaced uniformly on the mel scale,

$$m = 2595 \, \log_{10}\!\left(1 + \frac{f}{700}\right) \tag{4.3}$$

and the log energy of each filter output is taken,

$$e_j = \log\!\left(\sum_k H_j(k)\, |X(k)|^2\right), \qquad j = 1, \ldots, M \tag{4.4}$$

where H_j is the j-th triangular filter. A DCT-II decorrelates the log energies into the cepstral coefficients

$$c_i = \sum_{j=1}^{M} e_j \cos\!\left[\, i\,\left(j - \tfrac{1}{2}\right)\frac{\pi}{M}\right], \qquad i = 0, \ldots, 12 \tag{4.5}$$

of which the first **13** are retained as the frame's feature vector.

The classifier head is a small CNN ending in a softmax over the K event classes,

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \tag{4.6}$$

trained (Phase 1) by minimizing the cross-entropy loss

$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log p_k \tag{4.7}$$

against one-hot labels y. For deployment under TensorFlow Lite Micro the weights and activations are quantized to int8 with the affine map

$$x_q = \text{round}\left(\frac{x}{s}\right) + z \tag{4.8}$$

where s is the per-tensor scale and z the zero point. This step is intended to reduce memory use. The $<$50 ms frame deadline is a design budget, not a target-board timing result.

Model and deployment boundary. A trained four-class Stage-1 model (background, scream, cry and help) has been exported as an int8 TensorFlow Lite artifact of 34,920 bytes. On the held-out set described in Chapter 6, it achieved 86.62% accuracy, macro-F1 0.79 and weighted F1 0.86. The result supports the trained model and preprocessing pipeline; it is not a measurement of microphone distance, weather resistance, node power draw or real-street false alarms. The firmware retains a separate model-enabled build setting so a physical-node test cannot silently substitute a heuristic result for a model result.

The bench ESP32-S3 path is distinct. It uses a KY-037 analogue sound sensor and sends a signed sound-level candidate over local Wi-Fi to the Pi. Quiet-room false alarms have been observed, and the candidate does not directly authorize physical flight. A later sketch captures a short PCM clip so the Pi can attempt independent verification, but the documented hardware handoff says this version still needs upload and testing. Its outputs should not be labelled as the INMP441 classifier's measured performance.

4.3 Verification hub

The hub is a Python package on the Raspberry Pi 5 [32], one module per responsibility (Table 4.1).

The packet pipeline authenticates and checks replay freshness, resolves the registered node, and waits up to 8 s for the matching clip. With no clip, Stage-2 confirmation is unavailable:

Table 4.1: Hub functions and their responsibilities.

Function	Responsibility
Configuration	Stores the serial connection, security key, thresholds and drone service address.
Node registry	Maps each registered node to its surveyed location and records its latest accepted packet counter.
Packet security	Encrypts the alert, checks its authentication tag and rejects replayed packets.
Radio gateway	Reads alert packets from the ESP32 serial connection. A simulation mode supplies test packets.
Web service	Receives short audio clips and serves the incident map and alarm dashboard.
Audio verifier	Scores the clip using the available learned model. A development fallback cannot confirm an incident.
Evidence fusion	Combines confirmed audio with motion, light and time evidence to estimate severity.
Decision pipeline	Applies authentication, verification and severity checks before a dispatch request.
Dispatcher	Sends an approved request to the drone service using the surveyed node location.
Service entry point	Starts the serial gateway or its simulation mode.

the hub records the incident and makes no dispatch request. It does not substitute the Stage-1 score for missing evidence. With a clip, the hub runs the selected verifier and fuses its confirmed audio score with context. An automatic dispatch request requires a confirmed Stage-2 result and fused severity of at least 0.60. The verifier's threshold depends on its backend; 0.50 is not a universal current audio rule. Every incident, dispatched or not, is appended to the incident list that feeds the dashboard.

4.4 Authenticated field relay and telemetry gate

The October field-development implementation adds an explicit boundary between a remote incident report and the physical aircraft. The cloud edge relay stores signed reports and exposes separate operator, Pi/ESP-device and read-only diagnostic routes. The local receiver or ESP32 flight-relay firmware claims only an operator-approved, short-lived command. It then reads the local flight API or MAVLink telemetry before any trigger request is created. The browser cannot address MAVLink, arm a vehicle or write a motor output.

The preflight policy rejects a command when the mission queue is busy, the aircraft is armed, heartbeat or telemetry is stale, GPS lacks a fresh 3D fix with at least eight satellites, the reported home is not near the aircraft, RC readiness is absent, or the target violates the configured range/geofence limits. In its normal form it also requires known battery-cell count, plausible voltage and at least 30% reported battery. The firmware requests and decodes HEARTBEAT, position, GPS quality, system status, pilot receiver, home position, armed state and estimator health reports; the values are displayed as evidence, not guessed from a successful HTTP request.

The temporary no-battery profile is deliberately narrower than normal operation. It exists because battery telemetry was unavailable on the development platform and must be enabled explicitly. It retains GPS, home, RC and disarmed-state checks, caps API mission duration at 120 s, nominal requested altitude at 1 m, nominal requested speed at 0.5 m/s and the API geofence at 60 m; it omits the observation hover. It does not establish that flying without battery monitoring is safe, and it must be removed once a calibrated power module is available.

4.5 Alert authentication and replay protection

The threat model is simple: a spoofed packet launches a drone; an eavesdropped packet reveals an incident in progress; a replayed packet re-launches a drone at an attacker's chosen time. The wire format (25 bytes, which fits in one LoRa frame; Figure 4.2, full layout in Appendix B) answers all three:

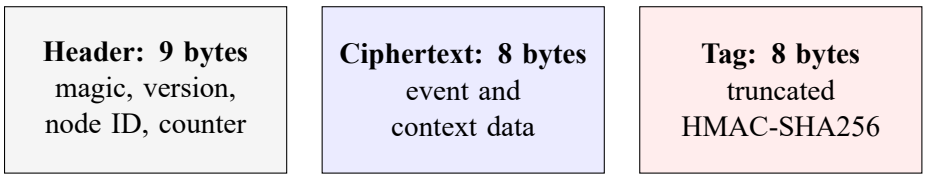


Figure 4.2: The sealed alert packet. The header is cleartext; the event data is encrypted and the packet is authenticated.

- **Confidentiality:** the 8-byte payload (event, confidence, PIR, light, battery) is AES-128-CTR encrypted. The CTR nonce is derived from the cleartext header (magic, version, node identifier, counter), unique per packet as long as the counter is monotonic.
- **Authenticity:** an 8-byte MAC (HMAC-SHA256 over header + ciphertext, truncated) using the per-node key. A bad MAC is dropped before decryption is trusted.

- **Replay protection:** the uint32 counter is monotonic per node; the hub rejects any counter $\leq$ the last accepted value for that node.
- **Key management:** each node uses a distinct key derived from the hub's master key and its registered identifier. The hub holds the master key; the node is provisioned with its derived key.

The node identifier and counter travel in cleartext so the hub can select the key and reject replayed alerts. Both are covered by the authentication tag.

Formally, the construction is as follows. Node n's key is derived from the hub's master key K_m as

$$K_n = \text{HMAC-SHA256}(K_m, \text{ID}(n))_{0:15} \tag{4.9}$$

where $\text{ID}(n)$ denotes the byte string consisting of "node:" followed by the node number in decimal, and the subscript $0{:}15$ selects the first 16 result bytes. The hub holds one master secret and each node uses its own derived key (HMAC per RFC 2104 [20]). The 8-byte payload P is encrypted in counter mode,

$$C = P \oplus E_{K_n}(\text{IV}) \tag{4.10}$$

where E is AES-128 [19] and the initial counter block IV is built from the cleartext header (magic, version, node identifier, counter). The IV is unique per packet as long as the node's counter is monotonic. The authentication tag is an encrypt-then-MAC over everything on the wire,

$$\text{tag} = \text{HMAC-SHA256}(K_n,\ \text{header} \parallel C)\,[0{:}8] \tag{4.11}$$

and the hub accepts a packet if and only if

$$\text{tag valid} \ \wedge\ \text{counter} > \text{counter}_{\text{last}} \tag{4.12}$$

for that node, bumping $\text{counter}_{\text{last}}$ only after acceptance. The truncated 8-byte tag leaves a blind-forgery success probability of 2^{-64} per attempt, which is negligible at LoRa alert rates; the trade-off would be revisited if the channel were ever widened.

4.6 Stage-2 audio verification

The current verifier returns a score and a separate confirmation decision. It examines overlapping one-second audio windows, spaced 0.25 s apart. Backend selection depends on model availability:

- **PANNs.** If its checkpoint loads, the default window threshold is 0.70 and at least three positive windows are required. This backend is intended for the Pi or a local installation.
- **YAMNet.** If PANNs is unavailable, the committed TFLite backend uses a default window threshold of 0.30 and three positive windows. A single very strong YAMNet window at or above 0.75 can also confirm a short event. These are software defaults rather than validated field settings.
- **Eligible prosodic check.** After the learned backend fails to confirm an eligible vocal event, an independent stress check may run for an exact emergency-word event or a vocal Stage-1 candidate with confidence at least 0.75. Its threshold is 0.55.
- **Energy heuristic.** When neither learned backend loads, a development scorer can still report loudness and spectral features. The current verifier marks this result unconfirmed, so energy alone cannot dispatch. The earlier Phase-0 rehearsal used this fallback under its historical software settings.

For completeness, the historical fallback scorer is defined here. Over the clip's samples $x[n]$, $n = 1, \ldots, N$, it computes the loudness as the root-mean-square level

$$\text{RMS} = \sqrt{\frac{1}{N}\sum_{n=1}^{N} x[n]^2} \tag{4.13}$$

and the spectral centroid (the "brightness" of the clip, high for screams)

$$C = \frac{\sum_k f_k\,|X(k)|}{\sum_k |X(k)|} \tag{4.14}$$

where $X(k)$ is the magnitude spectrum at frequency f_k. Together with a burstiness term (burst $\in [0, 1]$, from the envelope's peak-to-mean structure) the score is

$$\text{score} = 0.45\,\min\!\left(1, \frac{\text{RMS}}{0.15}\right) + 0.35\,\text{clip}\!\left(\frac{C - 400}{1600},\, 0,\, 1\right) + 0.20\,\text{burst} \tag{4.15}$$

i.e. full loudness credit at RMS ≥ 0.15 full scale, and centroid credit ramping linearly from 400 Hz to 2 kHz.

4.7 Evidence fusion

A night-time scream in a dark spot with motion nearby carries more weight than a daytime shout on a busy road. With a the Stage-2 audio score, c the Stage-1 confidence, $p \in \{0, 1\}$ the PIR motion flag, $L \in [0, 255]$ the LDR light level, and $n \in \{0, 1\}$ the night indicator (1 between 20:00 and 06:00), the fused severity is the weighted sum (Figure 4.3)

$$S = 0.60\,a + 0.15\,c + 0.10\,p + 0.08\,d + 0.07\,n, \qquad d = 1 - \frac{L}{255} \tag{4.16}$$

clamped to $[0, 1]$. Audio dominates by design; the context terms adjust the result. For the automatic audio route, the current hub requires a confirmed Stage-2 result and the severity gate:

$$\text{dispatch request} \iff \text{Stage-2 confirmed} \ \wedge \ S \geq 0.60 \tag{4.17}$$

The verifier applies its model-specific window threshold before returning confirmation. An accepted service request does not prove physical takeoff. A supervised operator test has a separate authorization path and still requires flight-readiness checks. Mission priority is set separately:

$$\text{priority} = \texttt{high} \iff S \geq 0.75 \ \vee \ (a \geq 0.6 \ \wedge \ p = 1) \tag{4.18}$$

otherwise normal.

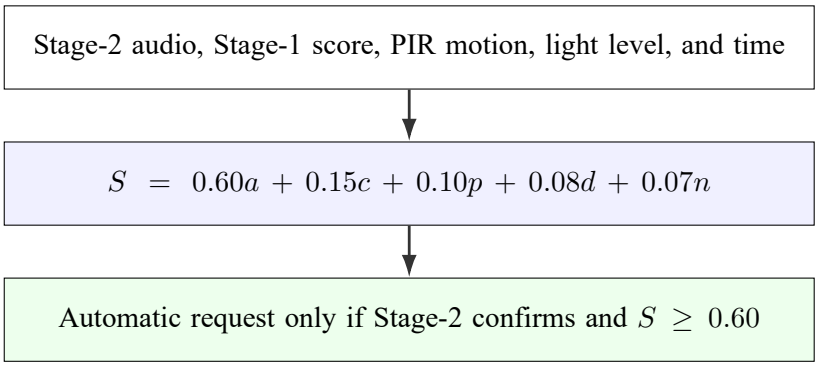


Figure 4.3: Evidence fusion and the automatic dispatch boundary. Prototype weights need calibration on measured field inputs.

For each alert, the hub records the audio result, Stage-1 confidence, motion, light and time contributions. The dashboard can show why the request passed or failed the gate. These prototype weights require calibration against measured bench and field data.

4.8 Dispatch decision

When an incident passes both gates, the hub sends the surveyed node location, event class and priority to the drone service. The flight service still checks the geofence, altitude limits and mission queue. The hub cannot bypass those safety checks.

CHAPTER 05

DESIGN AND IMPLEMENTATION

The response layer builds on the team's earlier flight-control work. The software was tested in simulation before the present sensing and hub architecture was added. Its mission checks remain relevant, but the earlier tests do not validate autonomous response by the current prototype. This chapter describes the flight sequence and the camera recording window shown in Figure 5.1.

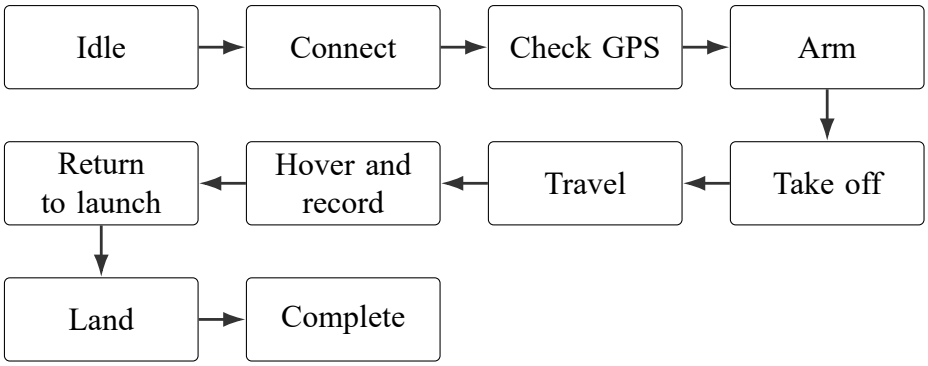


Figure 5.1: Normal mission sequence. A fault or cancellation enters guarded recovery.

5.1 Hazard analysis

At several points in the flight sequence, a delayed or incorrect command could leave the aircraft in an unsafe state. Table 5.1 lists the main hazards and the software check intended to handle each. These checks are part of the implementation; physical performance still requires supervised flight testing.

5.2 Verified mode transition

Every flight-mode change passes through one routine. It sends the requested mode, then checks the mode reported in the autopilot's heartbeat until the deadline. If the mode is still unconfirmed after 700 ms, the routine retries using the standard MAVLink command and the legacy mode command. Success means that the autopilot reports the requested mode, not merely that the client accepted the request.

If return-to-launch cannot be confirmed during an abort, the routine attempts landing; it also attempts return-to-launch when landing cannot be confirmed. This is a recovery attempt, not a guarantee of a safe landing. The check was motivated by an earlier ArduCopter 3.3 simulator test: the client reported success when a guided-mode request had not actually

Table 5.1: Flight hazards and the corresponding software checks.

ID	Failure	Risk	Software response
H1	Mode change ignored	Vehicle stays in the wrong mode.	Confirm the reported mode (Section 5.2).
H2	Emergency mode not accepted	Recovery command fails.	Retry and attempt the alternate recovery mode.
H3	New mission while airborne	A second takeoff is requested.	Block queue release until landing is confirmed.
H4	Brief GPS dropout	Unneeded emergency landing.	Debounce position loss before acting.
H5	Critical battery on return	Vehicle may not reach home.	Escalate the recovery action.
H6	Recovery severity reduced	Unsafe mode replaces a stronger response.	Never downgrade the selected action.
H7	Target outside geofence	Mission cannot finish safely.	Reject the target before launch.
H8	Route progress stalls	Battery drains while waiting.	Detect a stalled travel leg.
H9	Service stops during flight	Vehicle loses software supervision.	Request return before disconnecting.
H10	Unapproved dispatch request	Unauthorized flight.	Require access token and authenticated alerts.

changed the vehicle's mode. The flight service therefore uses the same confirmed routine for normal and emergency mode changes.

5.3 The landing interlock

Two mechanisms, redundant by intent (Figure 5.2). The **abort guarantee**: every abnormal termination (failsafe, recall, exception with an airborne vehicle) commands its action through Section 5.2 and then blocks, polling armed state and relative altitude, until disarm or a 240 s bound, before the queue regains control. The **pre-flight check**: a dequeued mission re-reads the armed flag and waits (bounded, 120 s) or fails rather than arm an armed vehicle. Together they make the invariant (the queue can never start a flight against an airborne vehicle) guaranteed by the design, not left to chance. The same property is defended at process boundaries: shutdown with an armed vehicle commands RTL (verified) before the MAVLink link drops (H9).

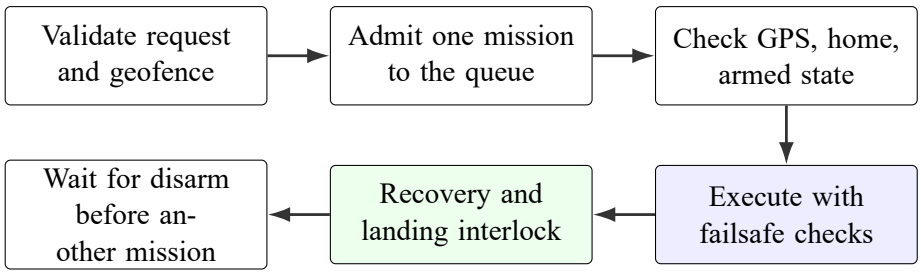


Figure 5.2: Earlier flight-stack interlocks. Each request passes readiness, in-flight checks, recovery, and disarm before queue reuse.

5.4 Failsafe arbitration

The arbiter evaluates four condition families at 1 Hz (battery low (20%) and critical (10%), GPS fix validity, geofence distance, mission wall-clock) and maintains exactly one demanded action under three rules. **Monotone severity:** demands only escalate (NONE $<$ RTL $<$ LAND); H6 closed by construction. **Debounce:** GPS loss fires only after N (default 3) consecutive bad samples, reset on recovery; it demands LAND because a return path is non-navigable without positioning; H4 closed with detection latency bounded at N seconds. **Fire-once:** each named hazard emits once per mission, with re-emission permitted only as escalation, so the incident log is not flooded with repeats. The executor couples to the arbiter inside every blocking loop, including the RTL loop, which re-reads the demand each second and swaps to LAND on escalation (H5).

Two pieces of mathematics underpin the arbiter's inputs. The geofence evaluator measures the vehicle's great-circle distance from home with the haversine formula: for latitudes φ_1, φ_2 and longitude difference $\Delta\lambda$,

$$d = 2R \arcsin \sqrt{\sin^2 \frac{\Delta\varphi}{2} + \cos\varphi_1 \cos\varphi_2 \sin^2 \frac{\Delta\lambda}{2}}, \qquad R = 6371 \text{ km} \tag{5.1}$$

This is the same computation used by edge validation (H7) and by the acceptance harness's closest-approach metric. The GPS-fix validity the arbiter debounces is itself the output of the autopilot's extended Kalman filter, which maintains the state estimate $\hat{x}$ through the standard predict/update pair

$$\hat{x}_{k|k-1} = F_k\, \hat{x}_{k-1|k-1}, \qquad P_{k|k-1} = F_k P_{k-1|k-1} F_k^{\top} + Q_k \tag{5.2}$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k \left(z_k - H_k\, \hat{x}_{k|k-1}\right), \qquad K_k = P_{k|k-1} H_k^{\top} \left(H_k P_{k|k-1} H_k^{\top} + R_k\right)^{-1} \tag{5.3}$$

fusing GPS, IMU, and barometer. The companion computer deliberately does not re-

implement this estimator; it consumes the autopilot's fix type and debounces it, trusting the EKF for fusion and itself for policy.

5.5 Hover-phase evidence recording

The recorder starts when the mission enters HOVERING and stops before return-to-launch, writing an mp4 tagged with the mission id into the mission log directory. This file is the evidence artifact for the incident record. On machines without a camera (SITL, dev laptops) it is a no-op stub, so the mission flow is byte-identical either way; on the airframe it drives the Pi Camera Module 3. Recording is mission-scoped by construction: the camera runs only over the incident, never in transit, and live streaming to police is explicitly future work (Chapter 7).

5.6 What the software refuses to own

Two authorities stay outside the stack by design. The hardware RC link (pilot mode-switch and kill) overrides anything this software commands; the last line of defence is hardware, not software. And ArduPilot's own pre-arm checks and firmware failsafes stay at stock values on hardware: the SITL-only relaxation is dead code unless simulation mode, and the documentation treats setting it on a real aircraft as a defect, not an option.

CHAPTER 06

TESTING AND SUMMARY OF RESULTS

6.1 Methodology

The project uses three levels of software validation. Each level checks a different boundary. Passing a unit test or simulated mission does not demonstrate safe physical flight or field detection accuracy.

Unit tests. The recorded 68-case inventory consists of 47 flight-control cases, 14 hub cases and 7 routing cases. The flight tests use a simulated vehicle to exercise failsafe escalation, GPS debouncing, geofence rejection, queue ordering, cancellation and landing interlocks. The hub tests check packet authentication, replay rejection, node lookup, severity fusion and the rule that an incident below the dispatch gate does not request a mission. The routing tests check obstacle keep-out geometry. These are software checks; they do not replace sensor, radio or airframe testing.

Simulation acceptance mission. The earlier flight harness starts ArduCopter simulation and the flight API, sends a test mission, and samples telemetry once per second. It checks connection, arming, takeoff, target approach, return and landing. The target is about 896 m from home, with a requested altitude of 15 m. The recorded approach and return limits are 5 m and 10 m respectively. Results from this harness apply to the earlier flight software, not automatically to the current cloud bridge.

Full-chain software rehearsal. A generated distress clip and a simulated sensing node exercise the sealed 25-byte alert, clip matching, hub scoring, fusion and mission request. The drone executes the mission in simulation and the camera recorder is a no-op. This rehearsal checks message flow and control sequencing. Its generated audio, fallback verifier, simulated physics and absent camera footage limit what can be concluded about a real incident.

6.2 Earlier flight software results

The team's earlier flight software completed six simulation missions on Windows 11 with Python 3.11.9. Table 6.1 summarizes the recorded acceptance metrics.

Table 6.1: Earlier software-in-the-loop acceptance missions.

Metric	Run 2	Run 3	Run 4	Run 5	Run 6
Duration (s)	503.7	321.6	321.3	330.7	331.3
Closest approach (m)	0.4	0.5	0.4	0.4	0.6
Distance from home (m)	0.1	0.0	0.0	0.0	0.0–0.2
Verdict	Pass	Pass	Pass	Pass	Pass

Run 2 exposed a test-predicate error that was corrected before the later runs. It did not change the recorded mission outcome (see Section 6.4).

Terminal accuracy of 0.4 to 0.6 m sits an order of magnitude inside the 5 m tolerance. It is bounded by the autopilot's loiter behaviour, not the dispatch layer. Wall-clock variance across runs 3 to 6 is $<3\%$, dominated by simulator boot.

6.3 Results: Phase-0 full-chain rehearsal

The rehearsal **passed** end to end. Chain trace, as logged:

- Sealed alert from simulated node (event scream, PIR active, dark LDR reading) authenticated, decrypted, and replay-checked by the real packet code; registry resolved the surveyed coordinates.
- In the earlier software snapshot, the energy fallback scored the synthesized clip above its then-used 0.50 gate. The current verifier does not count an energy-only score as confirmation.
- **Fused severity: 0.88** $\rightarrow$ priority high, clearing the 0.60 dispatch threshold.
- Dispatch POSTed; the earlier SITL mission exercised takeoff, transit, hover, return-to-launch, landing, and disarm. Its camera recorder was a no-op, so no footage was produced.
- Mission wall clock: **328 s**; closest approach 0.4 m; acceptance properties 8/8.

The reported acceptance run belongs to the earlier flight stack. It does not validate the newer cloud-to-Pi or ESP bridge.

These runs demonstrate specific software behavior, not a complete physical integration. Every interface a hardware phase will use (packet bytes, clip convention, thresholds, and trigger request) was exercised in its then-current software form. It deliberately does not prove acoustic detection performance (see Section 6.6).

6.4 Defect case studies (retained from v1)

Three defects found during v1 are retained as evidence for the design philosophy, because they now protect the whole VanniKawachh chain.

Case 1: the silent mode rejection (run 1). State machine reached TAKEOFF, altitude stayed 0.0 m, vehicle disarmed itself. Log forensics showed mode still STABILIZE despite the client reporting GUIDED. Root cause: unverified setter (H1). Fix: Section 5.2. Lesson: the autopilot's report is the only truth; the client's state is a wish. The two-stage acoustic pipeline applies the same lesson to Stage-1 claims.

Case 2: the racing test predicate (run 2). The harness inferred "connected" from a telemetry state string and raced a transition. Fix: an explicit boolean on /health. Lesson: tests must consume interfaces, not coincidences of internal state.

Case 3: the unguarded abort path (found by review, closed before it fired). The original abort used the bare setter (the exact call proven unreliable in Case 1) and returned with the vehicle airborne (H2+H3 compound). The unit tier now contains the regression tests that would have caught both. Lesson: the emergency path must be held to a higher standard than the nominal path, and it is the one most easily left to a lower one.

6.5 Acoustic-model and hub benchmark results

The latest Stage-1 model was evaluated on a held-out set of 1,719 clips. Table 6.2 reports the complete class-wise result rather than only the headline score. Its background false-alarm rate is 7.43%, and scream recall is 55.52%; those values are why Stage 1 is treated as a screening gate rather than an autonomous flight decision. The deployed int8 model artifact is 34,920 bytes.

Table 6.2: Stage-1 classification on 1,719 held-out clips.

Class	Precision	Recall	F1	Support
Background	0.916	0.926	0.921	1,386
Scream	0.661	0.552	0.603	281
Cry	0.537	0.947	0.686	38
Help	0.875	1.000	0.933	14
Macro average	0.747	0.857	0.786	1,719
Weighted average	0.866	0.866	0.864	1,719

The hub-side classifier has a separate speaker-disjoint internal benchmark using the production YAMNet TFLite representation and the the distress classifier feature route. On 218 untouched clips from the group-aware ASVP-ESD holdout, it achieved 97.25% accuracy, 97.18% balanced accuracy, 0.9679 macro-F1, 0.9420 distress precision and 0.9701 distress recall (TN/FP/FN/TP = 147/4/2/65). This is an encouraging internal benchmark,

not a source-external benchmark and not a field false-positive estimate. In particular, its four false positives rule out any claim of zero false triggers.

6.6 Measurements still needed

Stated plainly, because the credibility of the measured numbers depends on the honesty of the unmeasured ones:

- **No field acoustic result exists.** The Stage-1 and speaker-disjoint hub results in Section 6.5 are controlled dataset evaluations. The Stage-1 split is by source file and lacks a complete run manifest. Real microphone distance, latency, power draw, rain/wind/noise robustness and full-pipeline false-alarm rate remain unmeasured. The $<$50 ms Stage-1 value remains a design budget until it is measured on the exact ESP32-S3 hardware.
- **The present KY-037 bench route has false candidates.** Quiet-room false alarms were observed. A newer clip-sending sketch still needs upload and hardware validation. These observations are not results for the intended INMP441 model.
- **No external Stage-2 benchmark or field precision exists.** The internal 97.25% result uses a speaker-disjoint ASVP-ESD split; it must not be generalized to other voices, languages, streets or microphones. The Phase-0 score still used the labelled energy heuristic fallback for an integration test, not a measured field verifier.
- **No radio range or loss figures exist.** LoRa range (urban/open) and packet loss vs. spreading factor, and ESP-NOW clip-delivery reliability, are Phase-2 measurements.
- **No autonomous mission through the current cloud bridge is documented.** The Pi service is locked in the repository handoff, and its bench telemetry lacks usable GPS and battery data. The alternative ESP bridge is built but not flashed and does not arm or start AUTO. Physical aircraft photographs and earlier SITL runs do not establish a current node-to-aircraft mission. A staged path—props-off checks, power-module calibration, manual hover, supervised guided flight with RC override, then an outdoor VLOS integration test—is required before any operational claim.

These are the numbers the Phase 1 and 2 bench campaigns exist to produce, and the papers derived from this thesis will carry them only once measured.

Table 6.3: Evidence required before claiming an integrated field system.

Subsystem	Evidence available	Next acceptance record
Node audio	Offline model metrics; KY-037 candidates with observed false alarms	Flash the intended front end and log board version, model hash, latency, distance, noise conditions, false alarms, and misses.
Verification	Internal ASVP-ESD holdout and software thresholds	Use an independent source and field microphone; preserve clip IDs, speaker groups, confusion matrix, and threshold settings.
Alert transport	Packet and signed-request code	Measure radio range, loss, clip latency, authentication failures, and retry behavior with matched event identifiers.
Cloud and Pi	Queue and read-only telemetry checks; physical dispatch locked	Record a fresh grounded home, GPS fix, battery values, RC takeover, local gate decisions, and command lifetime.
Flight and footage	Earlier SITL missions and physical airframe photos	Supervised flight log plus time-stamped camera file that can be matched to one verified incident.

6.7 Safety, Privacy, and Legal Compliance

6.7.1 Privacy by construction

The intended fixed node processes ordinary audio frames locally and sends a short clip only after a candidate trigger. This is a data-minimization objective, not proof that the whole prototype avoids collecting personal information. The repository also includes phone and USB-microphone routes that upload WAV clips to a cloud service. The encrypted LoRa alert and the audio-clip path are separate; the repository does not demonstrate end-to-end encryption for every audio route. Recorded speech can contain sensitive information.

The cloud prototype uses SQLite for incident and command state, with deployment notes describing ephemeral storage. It does not yet establish a production retention schedule, deletion process, public notice, access-control policy, or dependable emergency record. A public-space study would need an approved collection protocol and a documented privacy review. The drone camera has a hover-scoped recording window in software, but usable footage from the reported simulation was not produced because its recorder was a no-op.

6.7.2 Alert integrity and remaining threats

The planned fixed-node LoRa packet uses a per-node key, a truncated HMAC tag, and a monotonic counter. The hub rejects malformed packets, unknown nodes, invalid tags, and stale counters before treating a packet as a fresh candidate. This narrows packet forgery and replay risks. It does not establish that a microphone heard a real emergency, that a captured node cannot be abused, or that radio jamming is prevented. A candidate still requires independent audio confirmation before automatic dispatch. Signed cloud reports and local telemetry gates add distinct checks on the newer development path.

6.7.3 Flight operations and pilot control

India's Drone Rules, 2021 require the intended flight area and its restrictions to be checked on Digital Sky before operation [21]. Any outdoor trial in this project is a supervised visual-line-of-sight test with an RC pilot able to take over. The repository's physical Pi service remains locked, and its recorded bench telemetry is insufficient for a guided mission. Registration, airspace authorization, pilot qualification, aircraft condition, and local permissions must be verified for the particular test rather than inferred from a software geofence or a prior simulation.

6.7.4 Dispatch restraint

An automatic audio incident requires a Stage-2 confirmation and sufficient fused severity before reaching the response queue. The local flight service also checks readiness, including fresh telemetry, a grounded home, GPS, battery policy, pilot readiness, and mission state. The separate operator real-mode request is a short-lived supervised test path; it bypasses audio verification only, not aircraft safety checks. A queued command, an API acknowledgement, and a flown mission are different evidence levels.

6.7.5 Radio-spectrum compliance

The intended LoRa installation requires a frequency, equipment, power, and duty-cycle plan consistent with the applicable Indian short-range-device rules. The cited WPC notification addresses low-power equipment in the 865–868 MHz band [23]; the 433 MHz SX1278 prototype radio cannot be presumed compliant under that notification. Before any public installation, the exact transceiver and antenna configuration must be checked against

current WPC conditions and equipment approvals. The thesis reports no measured range, packet-loss rate, or completed outdoor radio deployment.

CHAPTER 07

CONCLUSION

7.1 Conclusions

VanniKawachh integrates an infrastructure-based safety chain: local audio screening, authenticated event reporting, hub/relay verification, and a Pixhawk-controlled response platform. This thesis documents the whole path from the microphone and MFCC–CNN model to the LoRa packet, Raspberry Pi and ESP relay, MAVLink safety gate, F450 propulsion system and mission recovery state machine. Its essential engineering rule is that a claim is not a confirmation: a Stage-1 score is not an incident; a signed report is not a mission; a sent mode command is not a confirmed autopilot mode; and a return command is not a completed mission until landing and disarming are observed.

The evidence is correspondingly separated. Stage 1 reached 86.62% accuracy on 1,719 file-split evaluation windows, with only 55.52% scream recall. The separate hub-side feature classifier reached 97.25% accuracy on a 218-example speaker-disjoint internal holdout. The earlier software stack completed a recorded SITL acceptance run, while its Phase-0 rehearsal used synthesized audio and an energy heuristic. Neither model result estimates outdoor performance, and neither software run proves footage or autonomous operation through the current cloud bridges. The present KY-037 bench path has produced quiet-room false alarms, and physical dispatch remains locked in the documented Pi setup.

The next contribution must be measured integration: an actual microphone and board, independently checked verification, reliable transport, usable recording, and a supervised physical mission with fresh telemetry and RC takeover. Until then, the project is a well-specified prototype with useful component results, not a deployed emergency response system.

CHAPTER 08

FUTURE SCOPE

8.1 Future work

1. **Field measurement campaigns:** flash and time the measured Stage-1 model on the exact ESP32-S3 node; measure outdoor detection distance versus SNR, Stage-1 and Stage-2 latency, end-to-end false-positive rate on street noise, node energy use, and LoRa range/loss versus spreading factor; then tune fusion thresholds from those data.

2. **Live police streaming:** RTSP/WebRTC video from the drone to the dashboard over LTE, extending the evidence recorder of Section 5.5 into a real-time feed for responding officers.

3. **OpenCV victim tracking:** on-drone detection and framing of the person in distress during hover, keeping the camera and the aircraft usefully positioned without operator input.

4. **TDOA multi-node localization:** with synchronized clocks, time-difference-of-arrival across neighbouring nodes upgrades "which pole heard it" to a position estimate between poles, and provides the multi-node corroboration path sketched in Section 3.4.

5. **City-scale mesh:** many nodes per hub, many hubs per city; hub-to-hub LoRa or wired backhaul; fleet dispatch with per-airframe interlocks (the v1 multi-vehicle design carries over).

6. **Physical-flight readiness and modernization:** calibrate the power module and battery monitor, verify Pixhawk GPS/EKF/RC failsafes and MAVLink telemetry, complete props-off and manual-hover checks with a safety pilot, then conduct a bounded guided VLOS test before attempting an integrated response demonstration. In parallel: pymavlink [25] / MAVSDK migration, ArduPilot 4.x SITL validation and fault-injection campaigns.

Bibliography

[1] R. Pavithra, R. Ahalya, and C. Shuruthika, “Discreet AI-powered women’s safety app with audio recognition and emergency automation,” in *Proc. 2026 IEEE Students Conf. Eng. Syst. (SCEECS)*, Bhopal, India, 2026, doi: 10.1109/SCEECS68810.2026.11430037.

[2] Y. Kim, D. Jang, and S.-P. Lee, “Robust scream detection and scream temporal interval prediction using CNN-Transformer and windowing CNN,” *IEEE Access*, vol. 13, pp. 71374-71387, 2025, doi: 10.1109/ACCESS.2025.3556729.

[3] P. Sharma and T. J. Jebaseeli, “Smart scream and panic detection system for women using AI,” in *Proc. 2025 Int. Conf. Autom. Comput. Renew. Syst. (ICACRS)*, 2025, pp. 1554-1559, doi: 10.1109/ICACRS67045.2025.11324333.

[4] A. Srimathi, S. Jothi, and M. P. Sindhiya, “Human scream detection and analysis for crime reduction,” in *Proc. 2025 Int. Conf. Multidiscip. Sci. Comput. Intell. (ICMSCI)*, Erode, India, 2025, pp. 1528-1534, doi: 10.1109/ICMSCI62561.2025.10894058.

[5] P. S. Bharathi et al., “A novel IoT enabled women safety system design with emergency alert mechanism,” in *Proc. 2025 Int. Conf. Future Technol. Syst. (ICFTS)*, 2025, pp. 1-7, doi: 10.1109/ICFTS62006.2025.11031774.

[6] R. Snehith, S. Saranya, and N. R. Reddy, “A smart device for women safety using IoT,” in *Proc. 2025 Int. Conf. Intell. Syst. Sci. (ICISS)*, 2025, pp. 277-282, doi: 10.1109/ICISS63372.2025.11076270.

[7] Y. Kim, D. Jang, and J. Lee, “Development of scream detection system with large-scale scream dataset,” in *Proc. 2024 Int. Conf. Inf. Commun. Technol. Converg. (ICTC)*, Jeju, South Korea, 2024, pp. 590-593, doi: 10.1109/ICTC62082.2024.10826757.

[8] A. A. Fime, M. Ashikuzzaman, and A. Aziz, “Audio signal-based danger detection using signal processing and deep learning,” *Expert Systems with Applications*, vol. 237, art. no. 121646, Mar. 2024, doi: 10.1016/j.eswa.2023.121646.

[9] R. V. Hadkar et al., “An efficient IoT-enabled women safety device,” in *Proc. 2024 Int. Conf. Autom. Comput. Renew. Syst. (ICACRS)*, 2024, pp. 425-431, doi: 10.1109/ICACRS62842.2024.10841797.

[10] G. Uganya et al., “Smart women safety device using IoT and GPS tracker,” in *Proc. 2023 Int. Conf. Comput. Electron. Biomed. Syst. (ICCEBS)*, Chennai, India, 2023, pp. 1-6, doi: 10.1109/ICCEBS58601.2023.10449302.

[11] S. Potturi et al., “An SOS women safety device: Automatic emergency alerts and geo-location sharing with GSM/GPS integration,” in *Proc. 2026 Int. Conf. Intell. Comput. Netw. Syst. (IC-ICNS)*, Bhubaneswar, India, 2026, pp. 1-6, doi: 10.1109/IC-ICNS68863.2026.11537940.

[12] G. Ciaburro and V. Puyana-Romero, “Sound event detection in smart cities: A systematic review of methods, datasets, and applications,” *Big Data and Cognitive Computing*, vol. 10, no. 3, art. no. 83, 2026, doi: 10.3390/bdcc10030083.

[13] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, “PANNs: Large-scale pretrained audio neural networks for audio pattern recognition,” *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 28, pp. 2880-2894, 2020, doi: 10.1109/TASLP.2020.3030497; panns-inference package.

[14] S. B. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980.

[15] L. Sciullo, A. Trotta, and M. Di Felice, “Design and performance evaluation of a LoRa-based mobile emergency management system (LOCATE),” *Ad Hoc Networks*, vol. 96, art. 101993, 2020, doi: 10.1016/j.adhoc.2019.101993.

[16] R. David et al., “TensorFlow Lite Micro: Embedded machine learning for TinyML systems,” in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 3, 2021, pp. 800-811; documentation and the Micro Speech example, `https://www.tensorflow.org/lite/microcontrollers`.

[17] ArduPilot Development Team, “ArduPilot documentation” (firmware, SITL, and failsafe documentation), `https://ardupilot.org`; and MAVLink Development Team,

"MAVLink developer guide" (heartbeat telemetry and mode change commands), https://mavlink.io/en/. Accessed Jul. 2026.

[18] LoRa Alliance, "LoRaWAN L2 1.0.4 specification," LoRa Alliance Technical Committee, 2020. [Online]. Available: https://lora-alliance.org/resource_hub/lorawan-104-specification-package/.

[19] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, Nov. 2001, doi: 10.6028/NIST.FIPS.197.

[20] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF RFC 2104, Feb. 1997.

[21] Ministry of Civil Aviation, Government of India, "The Drone Rules, 2021," notification G.S.R. 589(E), *Gazette of India*, 25 Aug. 2021; DigitalSky platform (UIN registration and airspace zone map). https://digitalsky.dgca.gov.in. Accessed 2026-07-06.

[22] Government of India, "The Digital Personal Data Protection Act, 2023," Act No. 22 of 2023, *Gazette of India*, 11 Aug. 2023.

[23] Wireless Planning and Coordination Wing, Department of Telecommunications, Government of India: delicensing of the 865-867 MHz band for low-power wireless devices (G.S.R. 1048(E), 2005) and low-power short-range-device (SRD) provisions applicable to 433 MHz operation.

[24] DroneKit-Python 2.9.2. https://github.com/dronekit/dronekit-python. Accessed 2026-07-06.

[25] pymavlink. https://github.com/ArduPilot/pymavlink. Accessed 2026-07-06.

[26] dronekit-sitl 3.3.0. https://github.com/dronekit/dronekit-sitl. Accessed 2026-07-06.

[27] FastAPI. https://fastapi.tiangolo.com. Accessed 2026-07-06.

[28] React 18. https://react.dev. Accessed 2026-07-06.

[29] Leaflet 1.9. https://leafletjs.com. Accessed 2026-07-06.

[30] OpenStreetMap. https://www.openstreetmap.org. Accessed 2026-07-06.

[31] Pixhawk hardware reference. https://pixhawk.org. Accessed 2026-07-06.

[32] Raspberry Pi 5 product documentation. https://www.raspberrypi.com/documentation/. Accessed 2026-07-06.

[33] Espressif ESP32-S3 technical reference manual. https://www.espressif.com. Accessed 2026-07-06.

[34] Semtech SX1276/77/78/79 LoRa transceiver datasheet. https://www.semtech.com. Accessed 2026-07-06.

[35] InvenSense/TDK INMP441 omnidirectional I2S MEMS microphone datasheet. https://invensense.tdk.com. Accessed 2026-07-06.

[36] U.S. FAA, Part 107 (Small Unmanned Aircraft Systems). https://www.faa.gov/uas/commercial_operators. Accessed 2026-07-06.

Appendix A

Project Photographs

The photographs below show the project airframe in the team's development setting. The image with the aircraft aloft includes a tether and documents a supervised physical test, not an autonomous node-triggered response or verified camera evidence.



Figure A.1: Project quadcopter during a supervised tethered outdoor test.



Figure A.2: Project quadcopter and flight hardware before a test.

Appendix B

LoRa Alert Packet Wire Format

Each alert uses 25 bytes, as shown in Table B.1. The first 9 bytes identify the sender and event sequence. Eight encrypted bytes carry the observation, and the final 8 bytes verify that the packet has not been altered.

Table B.1: Fields in one 25-byte alert packet.

Bytes	Length	Purpose
0–1	2	Fixed packet marker, identifying this alert format.
2	1	Format version.
3–4	2	Node identifier, used to select the correct security key.
5–8	4	Increasing packet counter, used to reject replayed alerts.
9–16	8	Encrypted event and sensor values.
17–24	8	Authentication tag covering the header and encrypted values.

The encrypted section stores one byte each for the event class, confidence, motion flag, light level and battery percentage. Three bytes are reserved. Event values identify scream, call for help, cry or crash. The hub derives a separate key for each node from its master key and that node’s identifier. It rejects a packet with the wrong length or version, an unknown node, an invalid authentication tag or a counter that is not greater than the last accepted value.

Appendix C

Configuration Reference

Table C.1: Main hub settings and their purpose.

Setting	Default	Meaning
Master security key	Development value	Must be replaced before any deployment.
PANNs threshold	0.70	Three positive audio windows are required.
YAMNet threshold	0.30	Three positive audio windows are required.
Strong YAMNet window	0.75	One strong learned window can confirm the event.
Prosody threshold	0.55	Applies only when the vocal stress check is eligible.
Dispatch severity	0.60	Minimum fused severity after audio confirmation.
Clip wait time	8 seconds	Maximum wait for the matching clip; absence cannot confirm distress.
Gateway connection	COM3, 115200 baud	Serial port and speed of the ESP32 radio gateway.
Drone service	Local port 8000	Flight service address; its authentication token is unset in development.
Node and clip storage	Local files	Registry and received clip storage.
Clip server port	8990	Upload endpoint for short node recordings.

The flight service also configures the connection, home location, geofence, battery limits, GPS checks, stall timeout, queue limits and simulation mode. These settings are documented in the repository's system documentation.

Appendix D

Test Inventory

Tier 1: historical 68-case inventory. the flight-control unit suite (47): 10 failsafe-arbiter cases (thresholds, escalation, no-downgrade, debounce $\times 2$, fire-once, geofence, timeout, healthy baseline); 7 queue cases (priority, depth cap, prune, cancel $\times 3$, worker bookkeeping); 2 persistence cases; validation cases covering every rejection bound (coordinates, altitude, hover, priority, waypoints, including parametrized edge values); 3 configuration cases. the hub unit suite (14): packet seal/unseal round-trip; MAC tamper rejection; replay-counter rejection; registry lookup + unknown-node drop; fusion scoring and priority mapping; pipeline gating (no dispatch below verify/dispatch thresholds); degraded no-clip scoring; dispatcher payload shape. the obstacle-avoidance suite (7): keep-out routing geometry.

Tier 2 (the full-mission acceptance suite): the eight-property SITL acceptance flight of Section 6.1, runnable on any machine.

Tier 3 (the Phase-0 rehearsal script): the historical zero-hardware full-chain rehearsal of Section 6.3; its results are not a current physical mission or a camera-evidence demonstration.